DevOps Tips/Preparation for Assignment 1 (Part 2 of 2)

This document is designed to give you most of what you need to adapt parts of the Boto 3 example programs provided to complete the Assignment 1 core specification.

Downloading image from specified URL (Step 4)

Your program can download the image using a Python module such as *requests*. Another option is to call the `curl` command using `subprocess.run()`

Uploading files to your S3 bucket (Step 4)

We have already provided basic code for creating an S3 bucket:

```
s3.create_bucket(Bucket=bucket_name)
```

and for putting an object into an S3 bucket:

```
s3.Object(bucket_name, object_name).put(Body=open(object_name, 'rb')
```

You will need to enhance this to allow you to make objects publicly accessible (i.e. on the web). There are two steps:

1. Clear the default "block all public access" bucket setting; unfortunately this can only be done with an S3 client object.

```
s3client = boto3.client("s3")
s3client.delete_public_access_block(Bucket=bucket_name)
```

2. Set up a policy to make all objects added to the bucket public and apply this policy to your bucket.

```
bucket_policy = {
                "Version": "2012-10-17",
                "Statement": [
                {
                    "Sid": "PublicReadGetObject",
                    "Effect": "Allow",
                    "Principal": "*",
                    "Action": ["s3:GetObject"],
                    "Resource": f"arn:aws:s3:::{bucket_name}/*"
                }
                ]
}
s3.Bucket(bucket_name).Policy().put(Policy=json.dumps(bucket_policy))
```

(both of the above code snippets assume that you have a string variable called *`bucket_name`*).

You will need to create an index.html web page containing the required image loaded from your S3 bucket.

ContentType for objects (Step 4)

Adding a `ContentType` attribute to an object helps browsers to understand what to do with it. For example when putting your index.html file you can add the attribute `ContentType='text/html'`. For a jpg image it would be `ContentType='image/jpeg'`.
Otherwise your browser may download the object rather than displaying it.

Set up S3 static website hosting (Step 4)

You can set up static website hosting on an S3 bucket by creating a BucketWebsite resource. See this example:

```
website_configuration = {
    'ErrorDocument': {'Key': 'error.html'},
    'IndexDocument': {'Suffix': 'index.html'},
}

bucket_website = s3.BucketWebsite('my-bucket-name')

bucket_website.put(WebsiteConfiguration=website_configuration)
```

You will need to create an index.html web page containing the required image loaded from your S3 bucket.

subprocess.run() (Step 6 & potentially elsewhere)

It is often convenient to run shell commands from within Python. For example in step 6 you can do this to run the scp and ssh commands. There are many usage variations. The following example shows a basic use of it (at the Python interpreter)

```
$ python3
>>> import subprocess
>>> cmd = "ls -l"
>>> result = subprocess.run(cmd, shell=True)
-rw-rw-r--  1 jbloggs  jbloggs  0 Jan 16 10:37 testfile1
-rw-rw-r--  1 jbloggs  jbloggs  0 Jan 16 10:37 testfile2
-rw-rw-r--  1 jbloggs  jbloggs  0 Jan 16 10:37 testfile3
>>> print result.returncode
0
```

Note that you can check the return code which matches the exit code provided by the shell command executed.

Script permissions (Step 6)

If you copy a script up to an EC2 instance, you'll need to make it executable . The command for this will be something like:

```
ssh -i mykeyfile.pem ec2-user@52.42.23.2 'chmod 700 monitoring.sh'
```

SSH remote command execution (Step 6)

The general syntax for SSH remote command execution on an EC2 instance is:

```
ssh -i mykeyfile.pem ec2-user@ip_address 'enter command here'
```

You'll need the following additional option for the *first* SSH connection to a new instance:

```
-o StrictHostKeyChecking=no
```

(this is required to suppress the new host key confirmation (yes/no) prompt)

Putting the above together, your first ssh command might look something like this:

```
cmd1 = "ssh -o StrictHostKeyChecking=no -i  mykeyfile.pem
        ec2-user@" + ip_address + " 'pwd'"
```

(replace `'pwd'` with a more useful command).

Before executing a remote command from within a Python script, it's a good idea to print the command string to the console (to help with debugging). You should also print the status and output values for the same reason.

You may have noticed that the "yum" package manager on Amazon Linux prompts the user for (yes/no) confirmation before installing a package. This can be suppressed using the yum `-y` option. For example, to remotely install apache on an EC2 instance with IP address 52.42.23.2, you might have the following:

```
ssh -o StrictHostKeyChecking=no -i mykeyfile.pem ec2-user@52.42.23.2 './monitoring.sh'
```

SCP (Secure copy) (Step 6)

scp allows you to transfer files using SSH. The command to copy a file called monitoring.sh in the current directory on the local machine to an EC2 instance with IP address 52.42.23.2 is:

```
scp -i mykeyfile.pem monitoring.sh ec2-user@52.42.23.2:.
```

(notice the colon following by dot at the end, after the IP address or DNS name).

Create AMI (Step 7)

You can use the *instance.create_image()* function to create an AMI where *instance* is your EC2 instance object. To ensure that your user data script has completed the web server installation before you create the image, you will probably need your program to wait for some time. There are a few options for this, including the following:

- You could run time.sleep() for a period of time to "guess" how long it will take until the web server is ready, possibly in combination with a waiter function. A limitation of this is that applying OS patches (or other software installation) in User Data can take an unpredictable amount of time.
- You could check the output of the monitoring script from step 6 (or do something similar to check for the httpd process).

Advice on debugging your program

Build your program incrementally and test after each change. However, be careful to frequently terminate created instances that may build up in number while testing your code repeatedly. Include lots of calls to print() to show yourself what the program is doing – do not "fly blind"!

Note on resources and budget

As well as terminating instances when you no longer need them, make sure you specify *nano* instances at launch.